

UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE

DEPARTMENT OF COMPUTER SCIENCE

Object-Oriented Programming

INSPECTION

Group N° 5

Project: Bookly - BookStore

Team Name: 404 FOUNDERS

Members:

Anthony Marcelo Arevalo Sangucho

Dylan Jair Cumbajin Oña

Juan José Alvarez mina

Instructor: Edison Lascano PhD

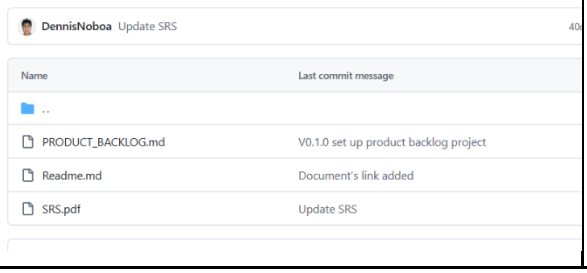
NRC: 36863

October 29, 2026

Quito – Ecuador

Group inspector: N°1

DEFINITION			
N #	Evidence	Observation	Grade
1	Invoicing and Transaction Processing Issuing purchase receipts manually requ out receipts by hand, which is time-consuming and inefficient during busy hour Spreadsheet tools lack native functionality to format and output customer-facin from raw row data. Bookly streamlines point-of-sale operations by automaticall item totals, applying pricing rules, and generating printed or digital invoices imi upon sale confirmation.	This statement is inaccurate. Spreadsheets can format and print invoices using templates. The comparison should explain the additional manual work required.	−0.1
2	**Problem** We need a system that allows managing the books available in a bookstore and reco the sales made, with the aim of maintaining proper control of the existing books, thei prices, and the sales carried out. To understand what the program must accomplish, it's first necessary to know a bit a books, their authors, categories, price, stock, and the sales made. The system will re simple and will be mainly focused on book management, stock control and sales rec	The problem statement describes the proposed solution but does not clearly identify the current operational difficulties and their consequences.	−0.1
3	For example, a book can have at that moment 10 copies, if customers b during the day, the remaining stock at the end of the day will be: 10 - 4 = Keeping stock uploaded is fundamental for a bookstore to know which b available or which ones need to be replaced.	The word “uploaded” should be “updated,” because the sentence refers to maintaining accurate stock information.	−0.1
Total Grade			9.7

LIST OF POSSIBLE OBJECTS			
N #	Evidence	Observation	Grade
1		The list of possible objects was not provided. Therefore, the requirement of at least 10 well-defined objects cannot be verified.	0/10
Total Grade			0/10

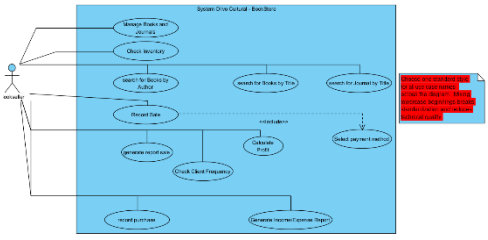
SRS IEEE 830			
N #	Evidence	Observation	Grade

2	**1. Introduction** • **Project Name:** Bookly • **Prepared by:** Group 5 - 404 Founders • **Version:** 1.0.0 • **Date:** 2026/10/21 **2. Overall Description** **2.1 Product Perspective** Bookly is a custom-built desktop application designed to help small bookst their daily operations. It replaces traditional methods like handwritten logbo general spreadsheet software, which can cause errors and take too much program works as an independent system that keeps all the store's data in focuses mainly on book management, stock control, and recording sales tr Bookly provides a simple and dedicated user interface, making it very easy bookstore staff to learn and use without needing complex technical knowle	The References subsection is missing.	−0.1
Total Grade			9.9/10

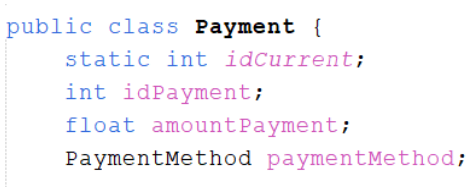
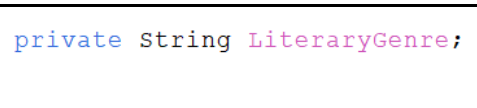
UML class diagram

N #	Evidence	Observation	Grade
1	-idSale : int -date : LocalDate -customer : Custumer -typeSale : String -payment : Payment	The class/type name is misspelled as Custumer. It should be Customer.	-0.1
Total Grade			9.9/10

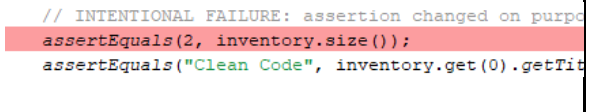
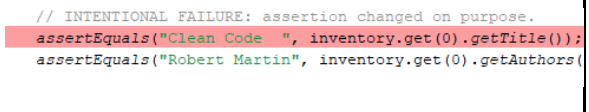
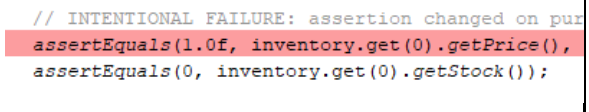
UML use case diagrams

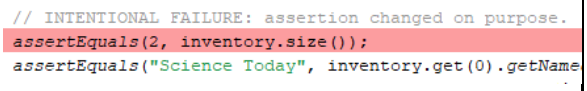
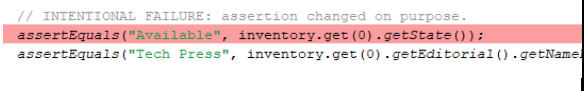
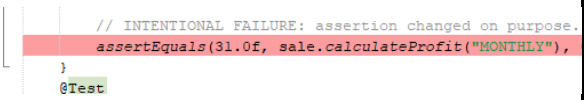
N #	Evidence	Observation	Grade
1		Choose one standard style for all use case names across the diagram . Mixing lowercase beginnings breaks standardization and reduces technical quality.	-0.1
Total Grade			9.9/10

Clean code

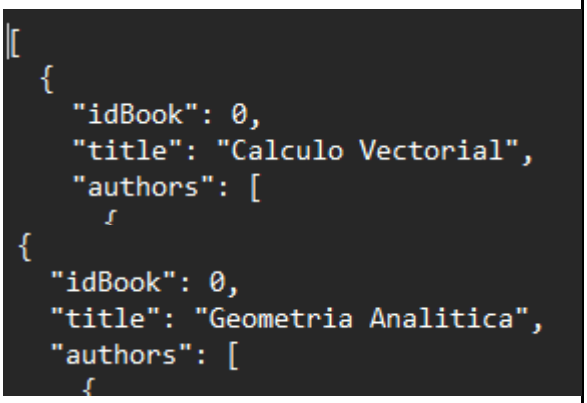
N #	Evidence	Observation	Grade
1		In `Payment.java`, fields are declared with package-private (default) access (e.g., `int idPayment;`, `static int idCurrent;`), violating encapsulation rules and the UML contract.	-0.1
2		LiteraryGenre.java, the field is defined as private String LiteraryGenre; (starting with an uppercase letter). This breaks Java's standard lowerCamelCase variable naming convention and confuses the variable name with the class type.	-0.1
3		In PaymentMethod.java, auto-generated code resulted in duplicate field printing	-0.1
Total Grade			9.7/10

Test cases/unit tests

N #	Evidence	Observation	Grade
1		Only one book is added to an empty inventory, but the test expects 2 records. The expected size should be 1. addBookAddsBookWithExpectedData()	-0.1
2		The test checks whitespace trimming but expects a title with trailing spaces. The expected title should be "Clean Code". addBookTrimsTitleAndAuthorNames()	-0.1
3		The book is created with a price of 0, but the assertion expects 1.0. The expected price should be 0.0. addBookAllowsZeroPriceAndStock()	-0.1

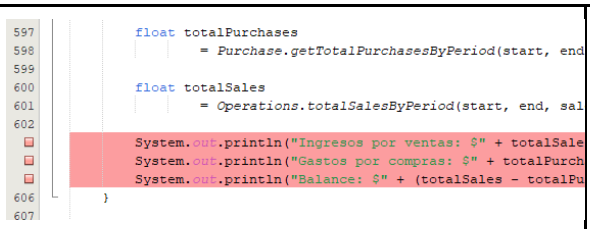
4		Only one journal is added to the inventory, but the test expects 2 records. The expected size should be 1. addJournalAddsJournalWithExpectedData()	-0.1
6		The journal is created with a price of 0, but the assertion expects 1.0. The expected price should be 0.0. addJournalAllowsZeroPriceAndStock()	-0.1
7		Thirty percent of $100 is $30, but the test expects $31. The assertion should expect 30.0 using the existing tolerance. calculateProfitReturnsThirtyPercentOfBookSaleTotal()	-0.1
Total Grade			9.3

Data files JSON

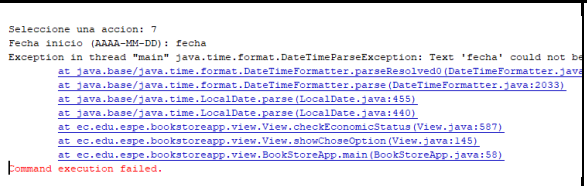
N #	Evidence	Observation	Grade
1		In books.json, different books share the same idBook: 0. After loading the saved data and adding another book, the new record also receives ID 0. The static idCurrent counter is not restored from the persisted records, so identifiers are reused across application sessions. Initialize the counter to the highest saved ID plus one after loading the inventory, or use persistent unique identifiers. Existing duplicate IDs must also be corrected while preserving any related references.	-0.1
Total Grade			9.9

Data reports: CSV

N #	Evidence	Observation	Grade

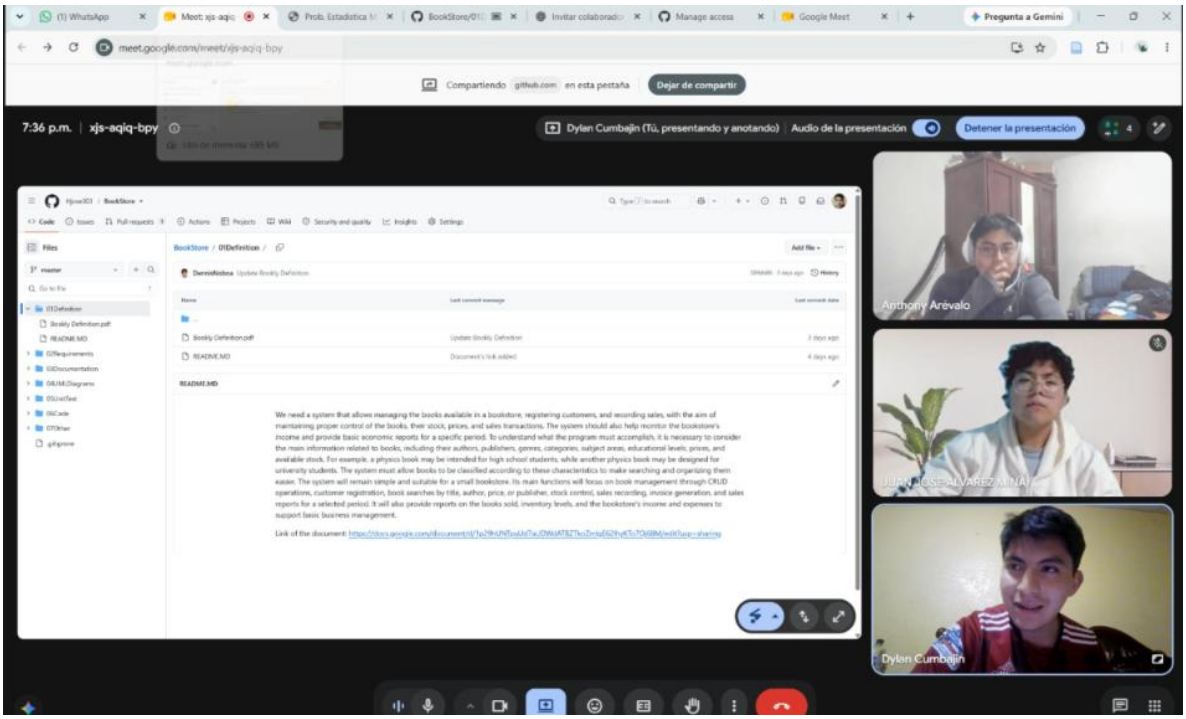
1		The application exports sales invoices to CSV, but the economic status and customer frequency reports are only displayed in the console. Only one CSV report type was identified. If the requirement means at least two distinct report types, an additional CSV export is missing. Add CSV export for the economic status report or another data report.	-0.1
Total Grade			9.9

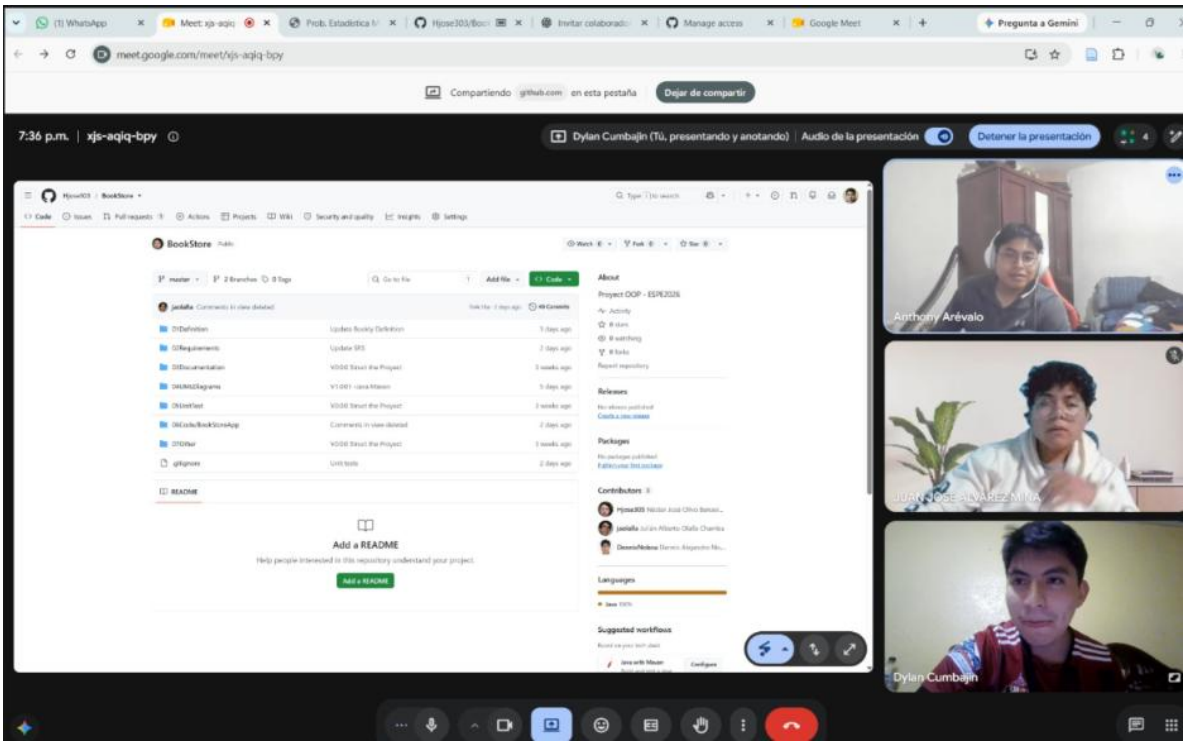
Execution of features

N #	Evidence	Observation	Grade
1		After purchasing 3 units of a book with an initial stock of 5, the purchase is recorded successfully, but the available stock remains 5 instead of increasing to 8. Update the inventory when the purchase is confirmed.	-1
2		Entering an invalid date in the economic status feature causes an unhandled DateTimeParseException and terminates the application. Validate the date and allow the user to correct it instead of closing the program.	-1

3	Nombre del proveedor: a 1.Comprar Libro 2.Comprar Revista Ingrese una Opcion: 1 0.Calculo Vectorial 1.Geometria Analitica Numero del libro: 999 Exception in thread "main" java.lang.IndexOutOfBoundsException: Index 999 out of bounds at java.base/jdk.internal.util.Preconditions.outOfBounds(Preconditions.java:100 at java.base/jdk.internal.util.Preconditions.outOfBoundsCheckIndex(Precondition at java.base/jdk.internal.util.Preconditions.checkIndex(Preconditions.java:302) at java.base/java.util.Objects.checkIndex(Objects.java:365) at java.base/java.util.ArrayList.get(ArrayList.java:428) at ec.edu.espe.bookstoreapp.view.View.addBookToPurchase(View.java:494) at ec.edu.espe.bookstoreapp.view.View.addPurchase(View.java:470) at ec.edu.espe.bookstoreapp.view.View.AddNewProducts(View.java:372) at ec.edu.espe.bookstoreapp.view.View.showChoseOption(View.java:139) at ec.edu.espe.bookstoreapp.view.BookStoreApp.main(BookStoreApp.java:58) Command execution failed.	Selecting book index 999 during purchase registration causes an unhandled IndexOutOfBoundsException and terminates the application. Check that the selected index is within the inventory range before accessing the list.	-1

Evidence:




meet.google.com/meet/xjs-aqiq-bpy
Compartiendo github.com en esta pestaña
Dejar de compartir
7:36 p.m. | xjs-aqiq-bpy
Dylan Cumbajín (Tú, presentando y anotando) | Audio de la presentación
Detener la presentación
BookStore
Add a README
Anthony Arévalo
JUAN JOSE ALVAREZ MINA
Dylan Cumbajín